

Analisis Exploratorio

Carlos de Leon

```

1 # Cargar bibliotecas Python
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 from sklearn.impute import SimpleImputer, KNNImputer
7 from sklearn.preprocessing import StandardScaler
8 import missingno as msno
9 import warnings
10 warnings.filterwarnings('ignore')

```

	id	edad	ingreso	educacion	experiencia	satisfaccion
1	1	28	35063	4	5	9
2	2	32	34401	5	14	6
3	3	54	49730	2	5	2
4	4	36	48017	5	6	NA
5	5	37	11760	3	15	1
6	6	56	65609	3	7	8

	id	edad	ingreso	educacion	experiencia	satisfaccion
0	1	22.0	38768	3	11	2.0
1	2	47.0	58514	3	14	10.0
2	3	38.0	60772	4	10	8.0
3	4	17.0	35009	4	4	3.0
4	5	28.0	57123	3	14	2.0

Analisis exploratorio inicial

```

1 # Resumen estadistico
2 summary(datos_r)

```

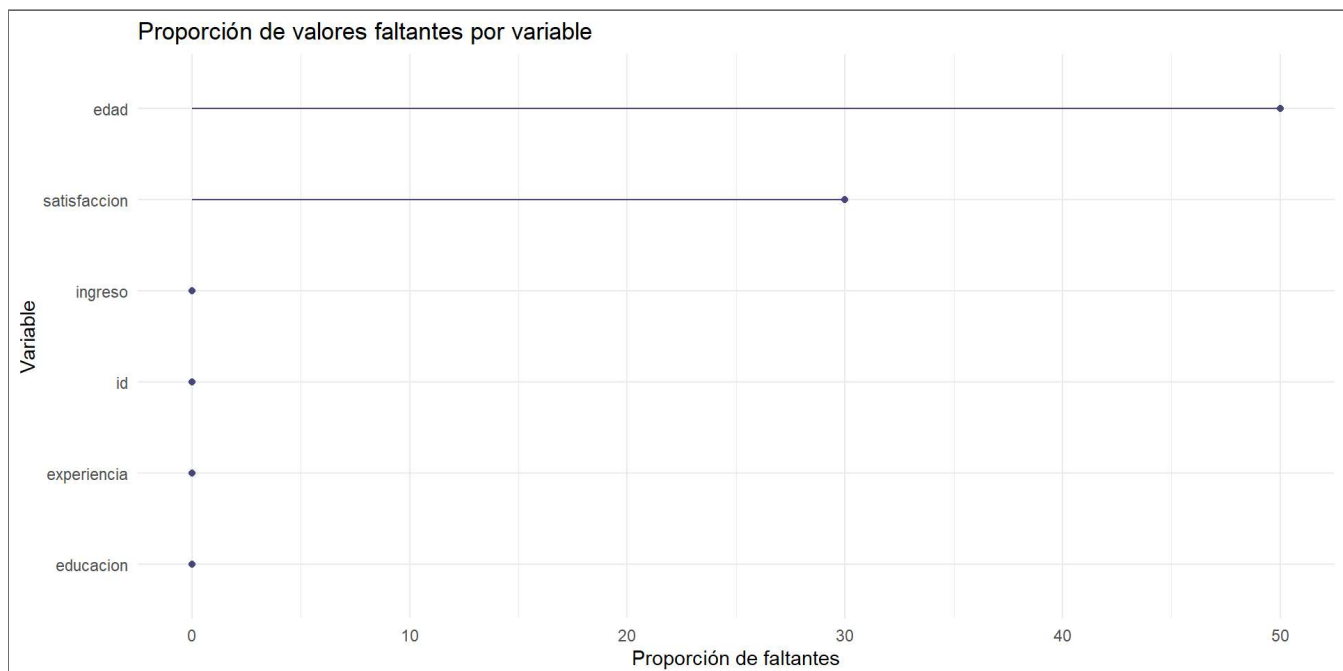
id		edad		ingreso		educacion	
Min.	: 1.0	Min.	: 1.00	Min.	: 4282	Min.	: 1.000
1st Qu.:	250.8	1st Qu.:	28.00	1st Qu.:	40462	1st Qu.:	2.000
Median	: 500.5	Median	: 35.00	Median	: 51094	Median	: 3.000
Mean	: 500.5	Mean	: 35.23	Mean	: 54907	Mean	: 3.164
3rd Qu.:	750.2	3rd Qu.:	43.00	3rd Qu.:	62080	3rd Qu.:	4.000
Max.	: 1000.0	Max.	: 67.00	Max.	: 416085	Max.	: 5.000
		NA's	: 50				
experiencia		satisfaccion					
Min.	: -12.00	Min.	: 1.000				

1st Qu.: 6.00	1st Qu.: 3.000
Median : 10.00	Median : 5.000
Mean : 10.01	Mean : 5.478
3rd Qu.: 15.00	3rd Qu.: 8.000
Max. : 34.00	Max. :10.000
	NA's :30

```
1 # Estructura del dataset
2 str(datos_r)
```

```
'data.frame':  1000 obs. of  6 variables:
 $ id      : int  1 2 3 4 5 6 7 8 9 10 ...
 $ edad    : num  28 32 54 36 37 56 41 20 27 30 ...
 $ ingreso : num  35063 34401 49730 48017 11760 ...
 $ educacion : int  4 5 2 5 3 3 5 3 2 1 ...
 $ experiencia : num  5 14 5 6 15 7 10 17 2 21 ...
 $ satisfaccion: num  9 6 2 NA 1 8 5 2 7 7 ...
```

```
1 # Visualización de datos faltantes
2 gg_miss_var(datos_r) +
3   labs(
4     title = "Proporción de valores faltantes por variable",
5     x = "Variable",
6     y = "Proporción de faltantes"
7   )
```



Equivalente en Python

```
1 # Resumen estadístico
2 print(datos_python.describe())
```

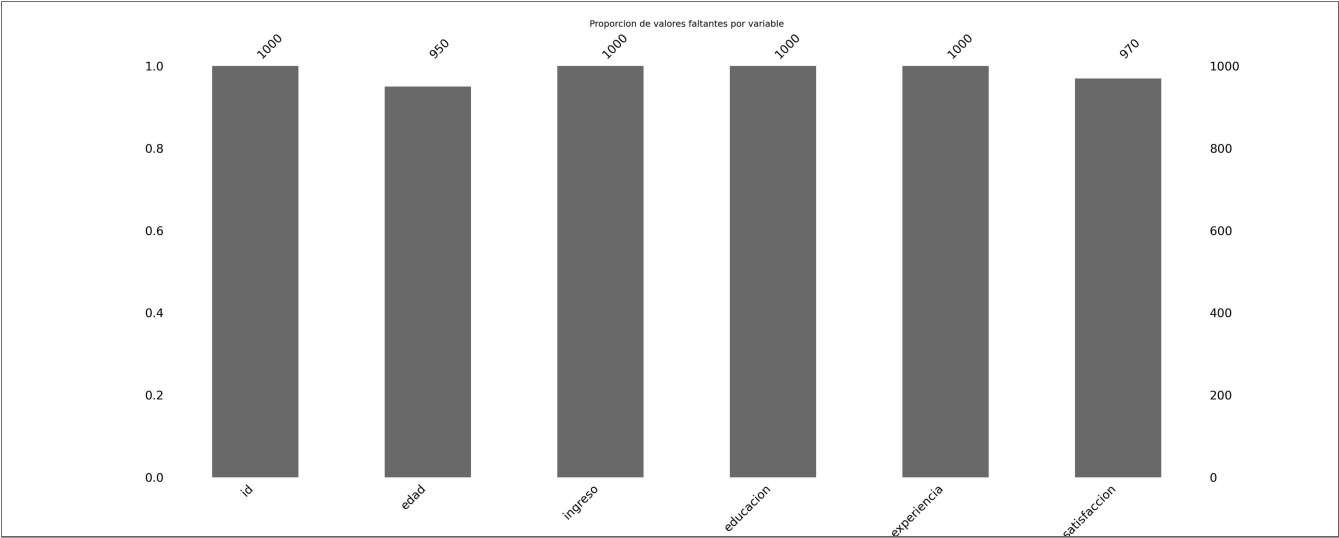
	id	edad	...	experiencia	satisfaccion
count	1000.000000	950.000000	...	1000.000000	970.000000
mean	500.500000	34.511579	...	10.159000	5.583505
std	288.819436	11.902633	...	6.874371	2.570319
min	1.000000	-4.000000	...	-9.000000	1.000000
25%	250.750000	27.000000	...	5.000000	3.000000
50%	500.500000	34.000000	...	10.000000	6.000000
75%	750.250000	43.000000	...	15.000000	8.000000
max	1000.000000	78.000000	...	31.000000	10.000000

[8 rows x 6 columns]

```
1 # Informacion del dataset
2 print(datos_python.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   id               1000 non-null   int64
1   edad             950 non-null    float64
2   ingreso          1000 non-null   int64
3   educacion        1000 non-null   int64
4   experiencia       1000 non-null   int64
5   satisfaccion     970 non-null    float64
dtypes: float64(2), int64(4)
memory usage: 47.0 KB
None
```

```
1 # Visualizacion de datos faltantes
2 plt.figure(figsize=(10,6))
3 msno.bar(datos_python)
4 plt.title("Proporcion de valores faltantes por variable")
5 plt.show()
```



Imputación de valores faltantes

Métodos de imputación

Tipos de imputación

1. Imputación simple: - Media / Mediana / Moda - Valor constante - Último valor observado (LOCF)
2. Imputación avanzada: - Regresión - K-Nearest (KNN) - Imputación múltiple por ecuaciones encadenadas (MICE) - Imputación por árboles de decisión

Imputación en R

```

1 # Método 1: Imputación por mediana
2 datos_r_mediana <- datos_r
3
4 datos_r_mediana$edad[is.na(datos_r_mediana$edad)] <-
5   median(datos_r_mediana$edad, na.rm = TRUE)
6
7 datos_r_mediana$satisfaccion[is.na(datos_r_mediana$satisfaccion)] <-
8   median(datos_r_mediana$satisfaccion, na.rm = TRUE)
9
10 # Método 2: Imputación por regresión lineal
11 modelo_edad <- lm(edad ~ ingreso + educacion + experiencia, data = datos_r)
12
13 datos_r_lm <- datos_r
14 indices_na_edad <- which(is.na(datos_r_lm$edad))
15
16 if (length(indices_na_edad) > 0) {
17   datos_r_lm$edad[indices_na_edad] <-
18     predict(modelo_edad, newdata = datos_r_lm[indices_na_edad, ])
19 }
20
21 # Método 3: Imputación múltiple (MICE)
22 imputaciones <- mice(datos_r, m = 5, method = "pmm", maxit = 50, seed = 123)

```

iter imp variable

```

1 1 edad satisfaccion
1 2 edad satisfaccion

```

```

1  3  edad  satisfaccion
1  4  edad  satisfaccion
1  5  edad  satisfaccion
2  1  edad  satisfaccion
2  2  edad  satisfaccion
2  3  edad  satisfaccion
2  4  edad  satisfaccion
2  5  edad  satisfaccion
3  1  edad  satisfaccion
3  2  edad  satisfaccion
3  3  edad  satisfaccion
3  4  edad  satisfaccion
3  5  edad  satisfaccion
4  1  edad  satisfaccion
4  2  edad  satisfaccion
4  3  edad  satisfaccion
4  4  edad  satisfaccion
4  5  edad  satisfaccion

```

```

1  datos_r_mice <- complete(imputaciones, 1)
2
3  # Comparación de métodos
4  comparacion <- data.frame(
5    Variable = c("Edad", "Satisfaccion"),
6    Original_NA = c(
7      sum(is.na(datos_r$edad)),
8      sum(is.na(datos_r$satisfaccion))
9    ),
10   Mediana_NA = c(
11     sum(is.na(datos_r_mediana$edad)),
12     sum(is.na(datos_r_mediana$satisfaccion))
13   ),
14   LM_NA = c(
15     sum(is.na(datos_r_lm$edad)),
16     sum(is.na(datos_r_lm$satisfaccion))
17   ),
18   MICE_NA = c(
19     sum(is.na(datos_r_mice$edad)),
20     sum(is.na(datos_r_mice$satisfaccion))
21   )
22 )
23
24 print(comparacion)

```

	Variable	Original_NA	Mediana_NA	LM_NA	MICE_NA
1	Edad	50	0	0	0
2	Satisfaccion	30	0	30	0

```
1 # Comparación de registros imputados
2 print(datos_r[indices_na_edad, ])
```

	id	edad	ingreso	educacion	experiencia	satisfaccion
63	63	NA	66742	4	10	5
70	70	NA	383435	4	14	5
73	73	NA	34617	4	6	3
107	107	NA	35217	1	14	4
121	121	NA	40352	5	18	2
126	126	NA	97761	1	12	4
164	164	NA	48272	3	10	9
178	178	NA	49849	3	7	1
293	293	NA	49423	3	10	8
309	309	NA	35839	2	9	4
340	340	NA	43422	3	16	3
377	377	NA	31067	3	14	6
380	380	NA	59310	2	-3	1
386	386	NA	20470	4	9	6
389	389	NA	60870	2	21	5
414	414	NA	62093	3	13	5
433	433	NA	57175	5	2	2
450	450	NA	49479	2	10	10
462	462	NA	60943	2	12	7
463	463	NA	57504	3	12	4
464	464	NA	59514	3	24	4

```
1 print(datos_r_mediana[indices_na_edad, ])
```

	id	edad	ingreso	educacion	experiencia	satisfaccion
63	63	35	66742	4	10	5
70	70	35	383435	4	14	5
73	73	35	34617	4	6	3
107	107	35	35217	1	14	4
121	121	35	40352	5	18	2
126	126	35	97761	1	12	4
164	164	35	48272	3	10	9
178	178	35	49849	3	7	1
293	293	35	49423	3	10	8
309	309	35	35839	2	9	4
340	340	35	43422	3	16	3
377	377	35	31067	3	14	6
380	380	35	59310	2	-3	1
386	386	35	20470	4	9	6
389	389	35	60870	2	21	5
414	414	35	62093	3	13	5
433	433	35	57175	5	2	2
450	450	35	49479	2	10	10

462	462	35	60943	2	12	7
463	463	35	57504	3	12	4

```
1 print(datos_r_lm[indices_na_edad, ])
```

	id	edad	ingreso	educacion	experiencia	satisfaccion
63	63	35.34437	66742	4	10	5
70	70	42.36677	383435	4	14	5
73	73	34.73247	34617	4	6	3
107	107	35.08423	35217	1	14	4
121	121	34.33904	40352	5	18	2
126	126	36.54905	97761	1	12	4
164	164	35.11556	48272	3	10	9
178	178	35.23493	49849	3	7	1
293	293	35.14149	49423	3	10	8
309	309	35.05070	35839	2	9	4
340	340	34.83863	43422	3	16	3
377	377	34.61620	31067	3	14	6
380	380	35.91478	59310	2	-3	1
386	386	34.32994	20470	4	9	6
389	389	35.27922	60870	2	21	5
414	414	35.34307	62093	3	13	5
433	433	35.16515	57175	5	2	2
450	450	35.33002	49479	2	10	10
462	462	35.53238	60943	2	12	7
463	463	35.26764	57504	3	12	4
464	464	34.97757	59514	3	24	4

```
1 print(datos_r_mice[indices_na_edad, ])
```

	id	edad	ingreso	educacion	experiencia	satisfaccion
63	63	40	66742	4	10	5
70	70	41	383435	4	14	5
73	73	16	34617	4	6	3
107	107	43	35217	1	14	4
121	121	35	40352	5	18	2
126	126	25	97761	1	12	4
164	164	20	48272	3	10	9
178	178	26	49849	3	7	1
293	293	12	49423	3	10	8
309	309	32	35839	2	9	4
340	340	30	43422	3	16	3
377	377	28	31067	3	14	6
380	380	39	59310	2	-3	1
386	386	38	20470	4	9	6
389	389	29	60870	2	21	5
414	414	32	62093	3	13	5
433	433	31	57175	5	2	2

450	450	57	49479	2	10	10
462	462	18	60943	2	12	7
463	463	26	57504	3	12	4
464	464	33	53511	3	12	4

Imputación en Python

```

1  datos_python = pd.DataFrame({
2      'id': range(1,n+1),
3      'edad': np.round(np.random.normal(35,12,n)).astype(int),
4      'ingreso': np.round(np.random.normal(50000, 15000, n)).astype(int),
5      'educacion': np.random.choice([1,2,3,4,5], n, p=[0.1, 0.2, 0.3, 0.25, 0.15]),
6      'experiencia': np.round(np.random.normal(10,7,n)).astype(int),
7      'satisfaccion': np.round(np.random.uniform(1,10,n))
8  })
9
10 from sklearn.linear_model import LinearRegression
11
12 # Método 1: Imputación por mediana
13 imputer_median = SimpleImputer(strategy='median')
14 datos_py_median = pd.DataFrame(imputer_median.fit_transform(datos_python),
15                                columns=datos_python.columns)
16
17 # Método 2: Imputación por KNN
18 imputer_knn = KNNImputer(n_neighbors=5)
19 datos_py_knn = pd.DataFrame(imputer_knn.fit_transform(datos_python),
20                             columns=datos_python.columns)
21
22 # Método 3: Imputación por regresión (solo para edad)
23 datos_py_lm = datos_python.copy()
24 edad_na_mask = datos_py_lm['edad'].isna()
25 if edad_na_mask.any():
26     train_data = datos_py_lm[~edad_na_mask]

```

	Variable	Original_NA	Mediana_NA	KNN_NA	LM_NA
0	Edad	0	0	0	0
1	Satisfacción	0	0	0	0

Detección y Tratamiento en Python

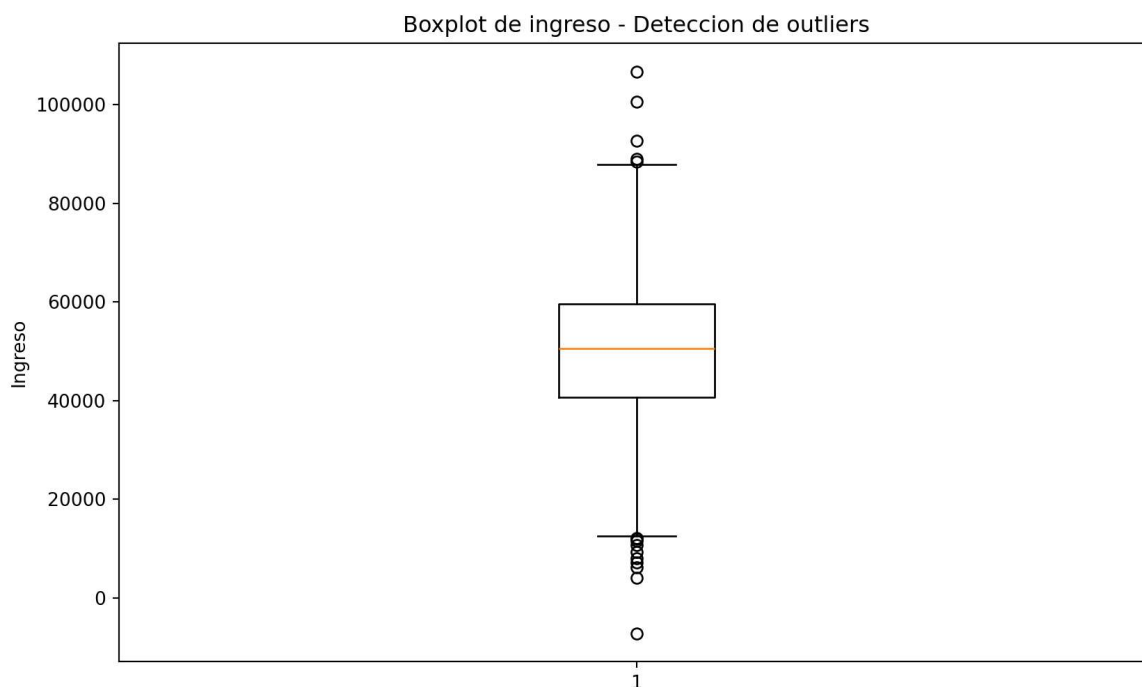
```
1 import matplotlib.pyplot as plt
2
3 # Función para detectar outliers usando IQR
4 def detectar_outliers_iqr(df, columna):
5     Q1 = df[columna].quantile(0.25)
6     Q3 = df[columna].quantile(0.75)
7     IQR = Q3 - Q1
8     lower_bound = Q1 - 1.5 * IQR
9     upper_bound = Q3 + 1.5 * IQR
10    return (df[columna] < lower_bound) | (df[columna] > upper_bound)
11
12 # Detectar outliers en ingreso
13 outliers_ingreso = detectar_outliers_iqr(datos_python, 'ingreso')
14 print(f"Número de outliers en ingreso: {outliers_ingreso.sum()}")
```

Número de outliers en ingreso: 15

```
1 # Boxplot para visualizar datos usando matplotlib
2 plt.figure(figsize=(10, 6))
3 plt.boxplot(datos_python['ingreso'])
```

```
{'whiskers': [<matplotlib.lines.Line2D object at 0x000002318601C7D0>,
<matplotlib.lines.Line2D object at 0x000002318601C910>], 'caps':
[<matplotlib.lines.Line2D object at 0x000002318601CA50>, <matplotlib.lines.Line2D
object at 0x000002318601CB90>], 'boxes': [<matplotlib.lines.Line2D object at
0x000002318601C690>], 'medians': [<matplotlib.lines.Line2D object at
0x000002318601CCD0>], 'fliers': [<matplotlib.lines.Line2D object at
0x000002318601CE10>], 'means': []}
```

```
1 plt.title('Boxplot de ingreso - Detección de outliers')
2 plt.ylabel('Ingreso')
3 plt.show()
```



```

1 # Tratamiento de outliers: Winsorización
2 def winsorizar(serie, lower_percentile=0.05, upper_percentile=0.95):
3     lower = serie.quantile(lower_percentile)
4     upper = serie.quantile(upper_percentile)
5     return serie.clip(lower=lower, upper=upper)
6
7 datos_py_winsor = datos_python.copy()
8 datos_py_winsor['ingreso'] = winsorizar(datos_py_winsor['ingreso'])
9
10 # Comparación antes/después de winsorización
11 print("Estadísticos originales:")

```

Estadísticos originales:

```

1 print(datos_python['ingreso'].describe())

```

```

count      1000.000000
mean       50301.089000
std        15279.820086
min        -7249.000000
25%        40639.250000
50%        50611.500000
75%        59597.000000

```

```
max      106769.000000
Name: ingreso, dtype: float64
```

```
1 print("\nEstadísticos después de winsorización:")
```

Estadísticos después de winsorización:

```
1 print(datos_py_winsor['ingreso'].describe())
```

```
count      1000.000000
mean       50358.100000
std        13711.609942
min        25541.350000
25%        40639.250000
50%        50611.500000
75%        59597.000000
max        76213.250000
Name: ingreso, dtype: float64
```

```
1 # Función para detectar outliers usando IQR
2 def detectar_outliers_iqr(df, columna):
3     Q1 = df[columna].quantile(0.25)
4     Q3 = df[columna].quantile(0.75)
5     IQR = Q3 - Q1
6     lower_bound = Q1 - 1.5 * IQR
7     upper_bound = Q3 + 1.5 * IQR
8     return (df[columna] < lower_bound) | (df[columna] > upper_bound)
9
10 # Detectar outliers en ingreso
11 outliers_ingreso = detectar_outliers_iqr(datos_python, 'ingreso')
12 print(f"Número de outliers en ingreso: {outliers_ingreso.sum()}")
```

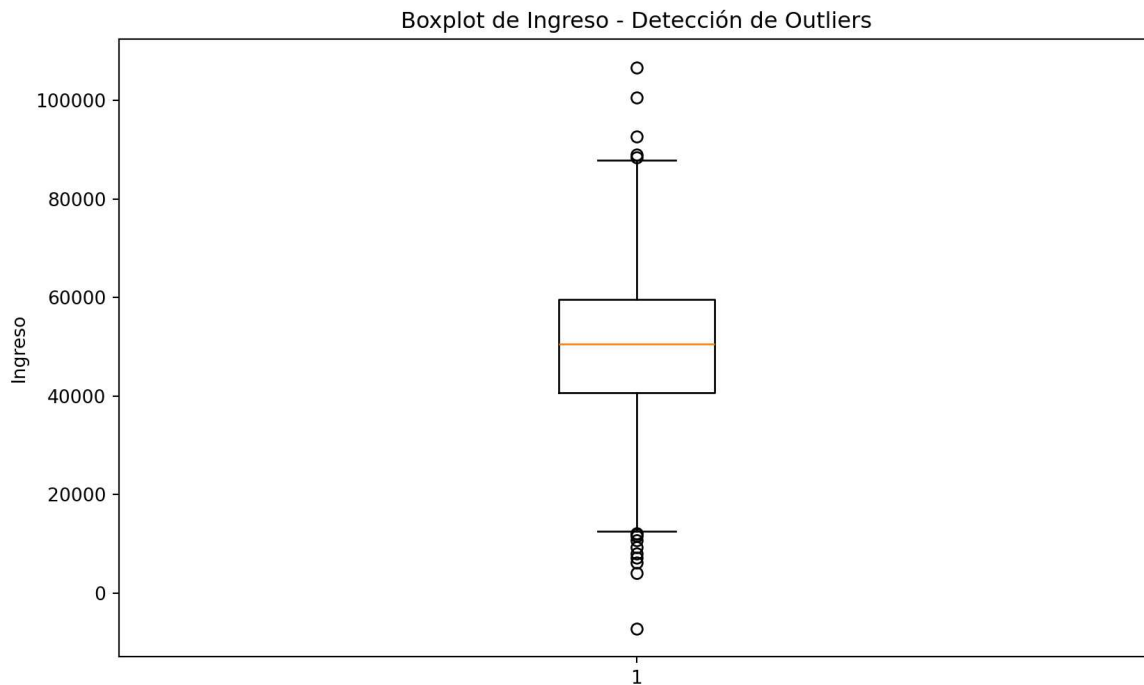
Número de outliers en ingreso: 15

```
1 # Boxplot para visualizar outliers
2 plt.figure(figsize=(10, 6))
3 plt.boxplot(datos_python['ingreso'])
```

```
{'whiskers': [<matplotlib.lines.Line2D object at 0x000002318604FED0>,
<matplotlib.lines.Line2D object at 0x00000231860AC050>], 'caps':
[<matplotlib.lines.Line2D object at 0x00000231860AC190>, <matplotlib.lines.Line2D
```

```
object at 0x00000231860AC2D0>], 'boxes': [<matplotlib.lines.Line2D object at
0x000002318604FD90>], 'medians': [<matplotlib.lines.Line2D object at
0x00000231860AC410>], 'fliers': [<matplotlib.lines.Line2D object at
0x00000231860AC550>], 'means': []}
```

```
1 plt.title('Boxplot de Ingreso - Detección de Outliers')
2 plt.ylabel('Ingreso')
3 plt.show()
```



```
1 # Tratamiento de outliers: Winsorización
2 def winsorizar(serie, lower_percentile=0.05, upper_percentile=0.95):
3     lower = serie.quantile(lower_percentile)
4     upper = serie.quantile(upper_percentile)
5     return serie.clip(lower=lower, upper=upper)
6
7 datos_py_winsor = datos_python.copy()
8 datos_py_winsor['ingreso'] = winsorizar(datos_py_winsor['ingreso'])
9
10 # Comparación antes/después de winsorización
11 print("Estadísticos originales:")
```

Estadísticos originales:

```
1 print(datos_python['ingreso'].describe())
```

```
count      1000.000000
mean       50301.089000
std        15279.820086
min        -7249.000000
25%        40639.250000
50%        50611.500000
75%        59597.000000
max        106769.000000
Name: ingreso, dtype: float64
```

```
1 print("\nEstadísticos después de winsorización:")
```

Estadísticos después de winsorización:

```
1 print(datos_py_winsor['ingreso'].describe())
```

```
count      1000.000000
mean       50358.100000
std        13711.609942
min        25541.350000
25%        40639.250000
50%        50611.500000
75%        59597.000000
max        76213.250000
Name: ingreso, dtype: float64
```

Analisis de resultados y visualizacion

Comparacion de metodos de imputacion

Speaker notes